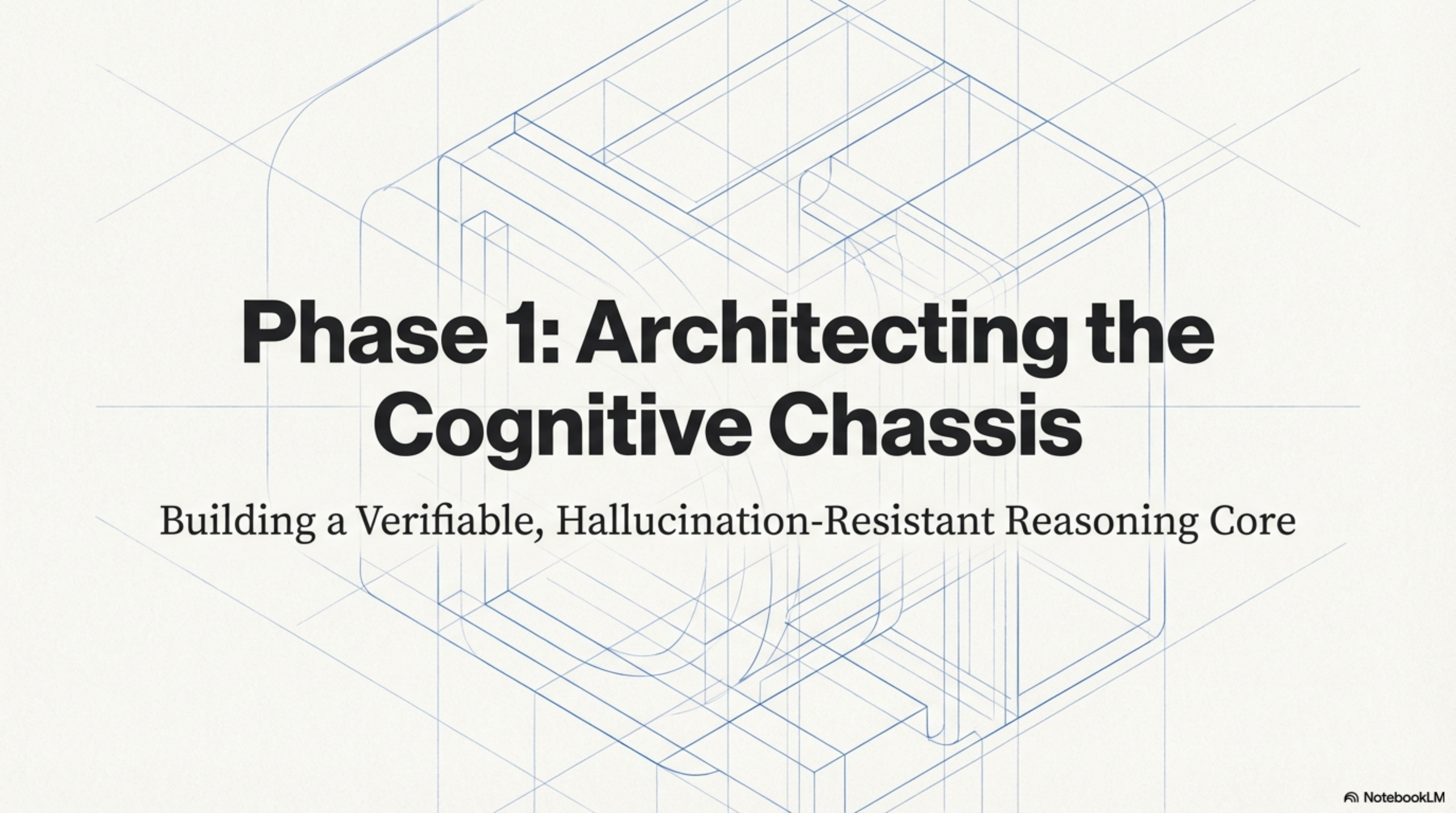


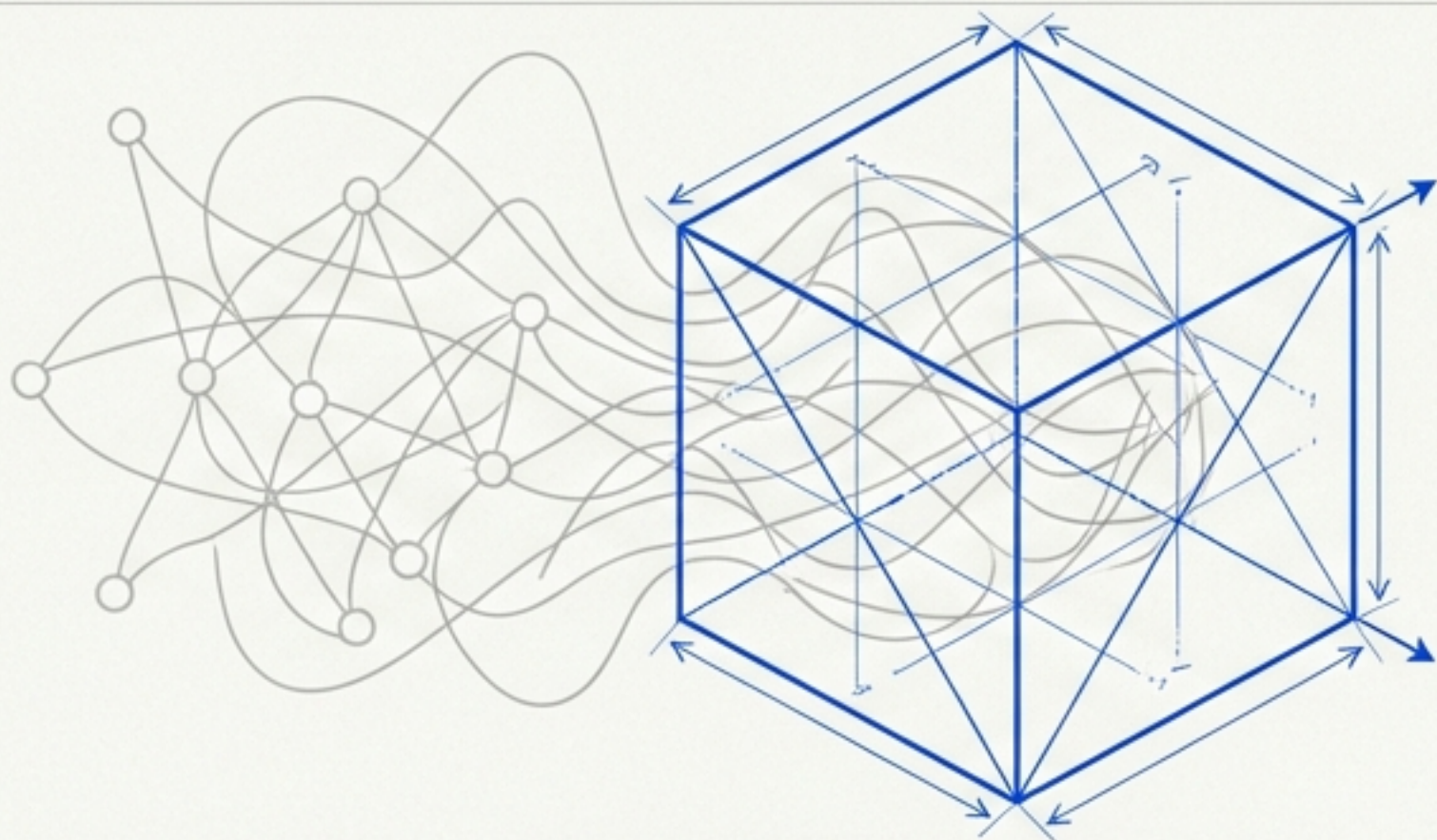


Phase 1: Architecting the Cognitive Chassis

Building a Verifiable, Hallucination-Resistant Reasoning Core

The Objective: Shackle Neural Creativity to Symbolic Rigor

The Problem

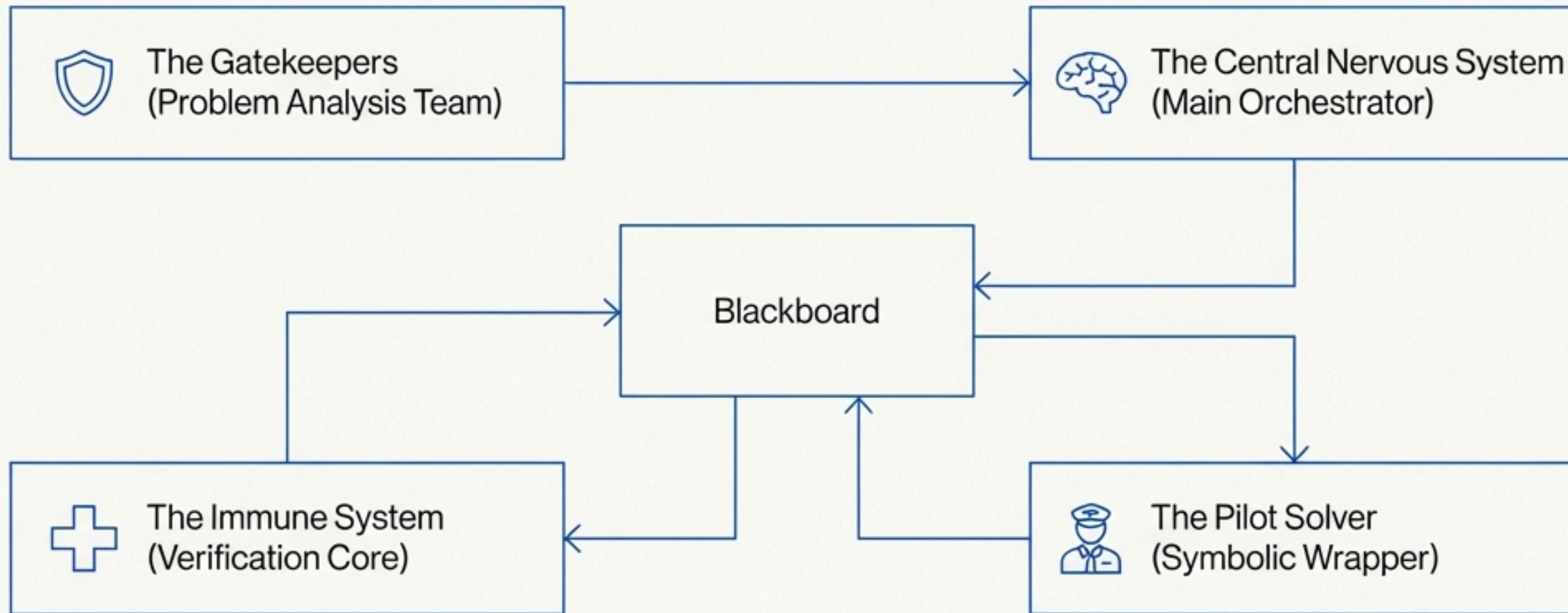


Standard AI systems “hallucinate,” generating plausible but incorrect answers. This makes them unreliable for high-stakes domains like mathematics and formal reasoning.

The Phase 1 Mission

- To construct the foundational **Cognitive Chassis**.
- The goal is not to solve *every* problem, but to build a system that can *legally* and *verifiably* solve a single problem without hallucination.
- This phase moves from the static infrastructure of Phase 0 to the first dynamic “heartbeat” of the system: the **Orchestrator-Prover-Verifier** loop.

The Architecture of a Query's Journey



Introducing Our Test Casplay Pro 65 Medium

To understand this system, we will follow the journey of a single query from start to finish:

Calculate the derivative of $x^2 + 1$

Step 1: The Gatekeepers Sanitize Chaos into Structure

Objective: “Prevent ‘Garbage In, Garbage Out’ by translating ambiguous language into strict mathematical objects before they enter the reasoning core.”

Component 1: Syntax Parser & Autoformalizer

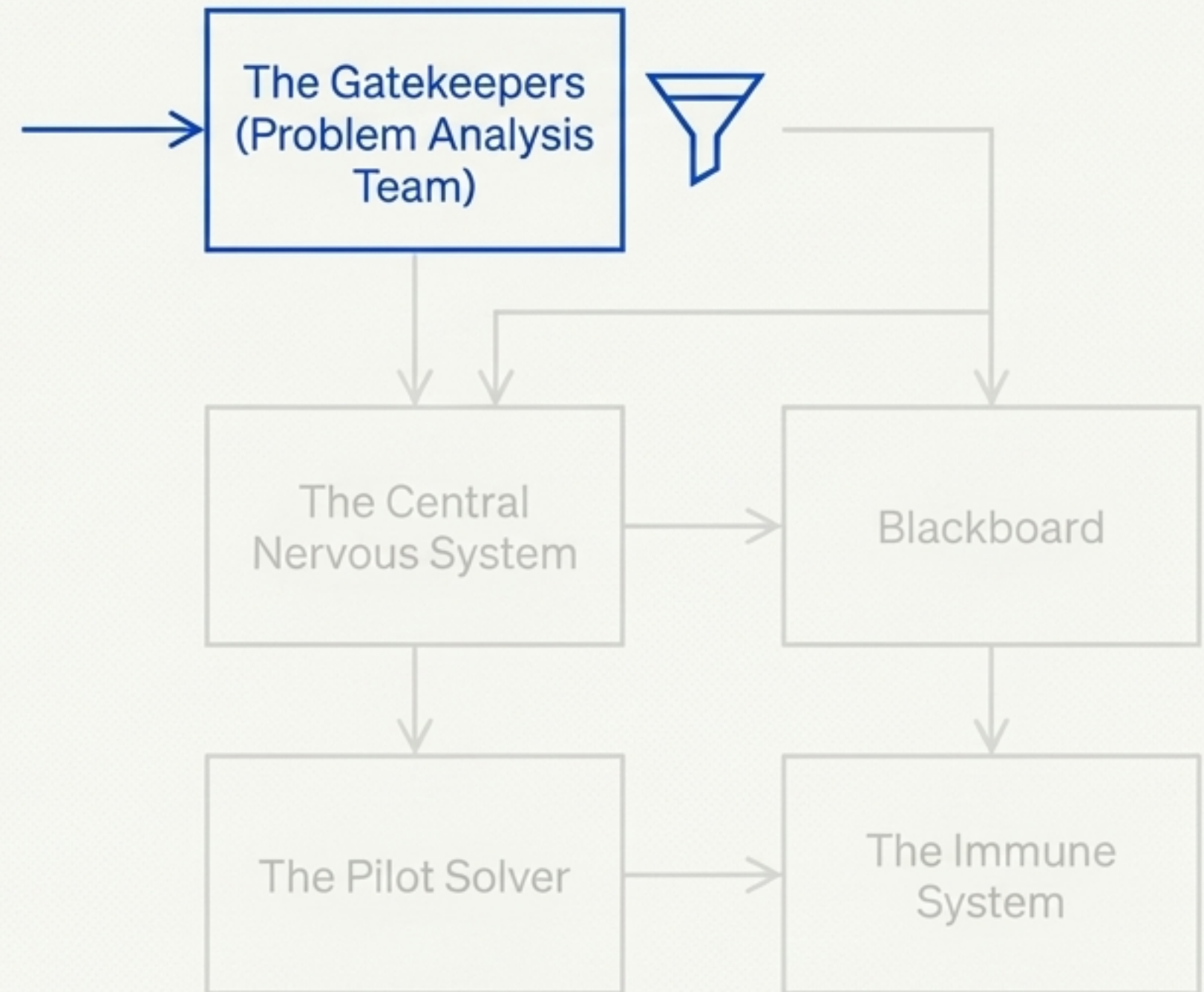
Function: A “transducer” agent that translates natural language and LaTeX into the **OMDoc/OpenMath** standard.

Example: Converts “Find the derivative of x squared” into a structured expression tree.

Component 2: Structure Recognizer

Function: A meta-classifier that tags the problem type. It does not solve; it categorizes.

Categories: **Computation** (“Calculate this”), **Proof** (“Show that...”), and **Optimization** (“Find the maximum...”). This tag is crucial for routing.



Step 2: The Orchestrator Manages; It Never Solves

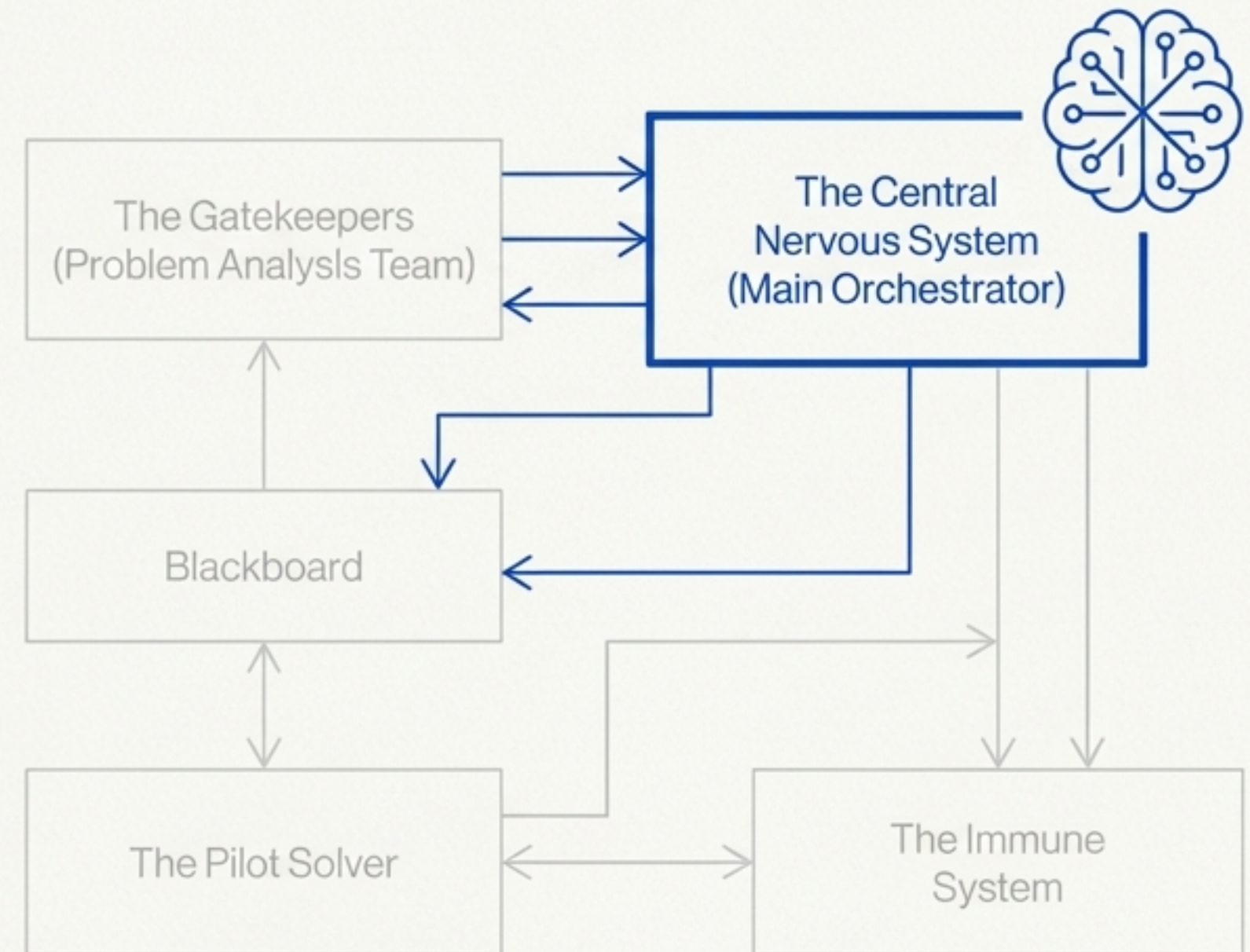
Objective: Instantiate the Tier 1 agent, the “General Contractor” whose sole job is management. It enforces the strict separation of **Problem Understanding** from **Problem Solving**.

Key Capabilities:

- **Decomposition Engine:** Uses **Hierarchical Task Network (HTN)** logic to break a request into sub-goals (e.g., ‘First simplify, then integrate’).
- **Routing Logic:** Connects to the **Directory Facilitator (DF)** to find agents with the required capability (e.g., ‘Calculus’) based on the problem tag. This ensures the system is extensible.

The Core Constraint: The “Non-Intervention” Directive

The Orchestrator is strictly forbidden from performing calculations. If it sees ‘2+2’, it *must* delegate, never compute.



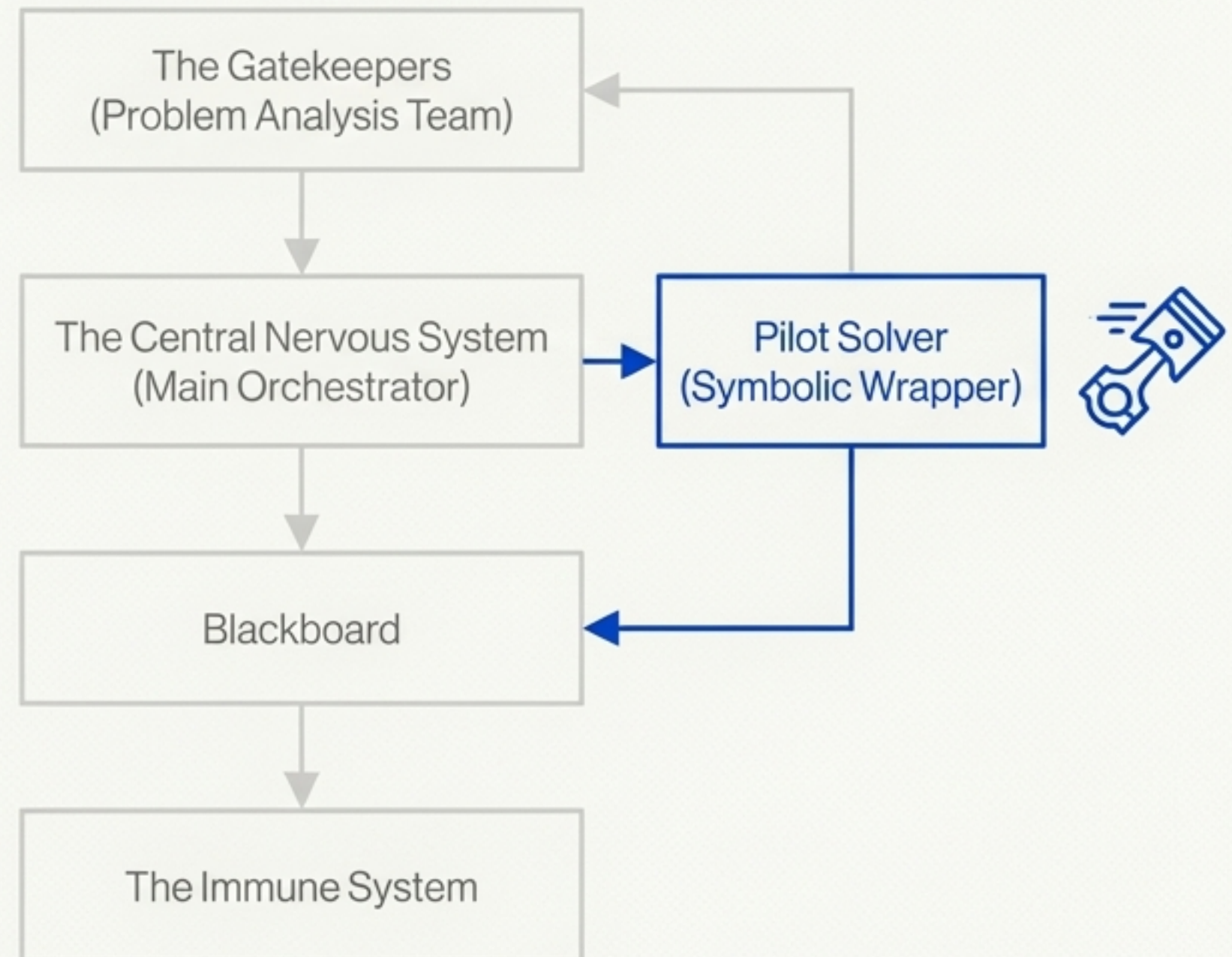
Step 3: The Pilot Solver Acts as the System's "Tracer Bullet"

Objective: "Create a temporary 'stand-in' for the future agent workforce to test the entire pipeline."

The Agent: SymPy Wrapper

- Encapsulates a robust Computer Algebra System (**SymPy**) inside a standard agent shell.
- Acts as the "Prover" or "Executor."
- It receives OMDoc inputs, executes deterministic symbolic operations (like `sympy.diff`), and posts the raw result back to the Blackboard.

Strategic Purpose: "To prove that a mathematical payload can travel from User → Orchestrator → Blackboard → Solver and back, verifying the system's core functionality."



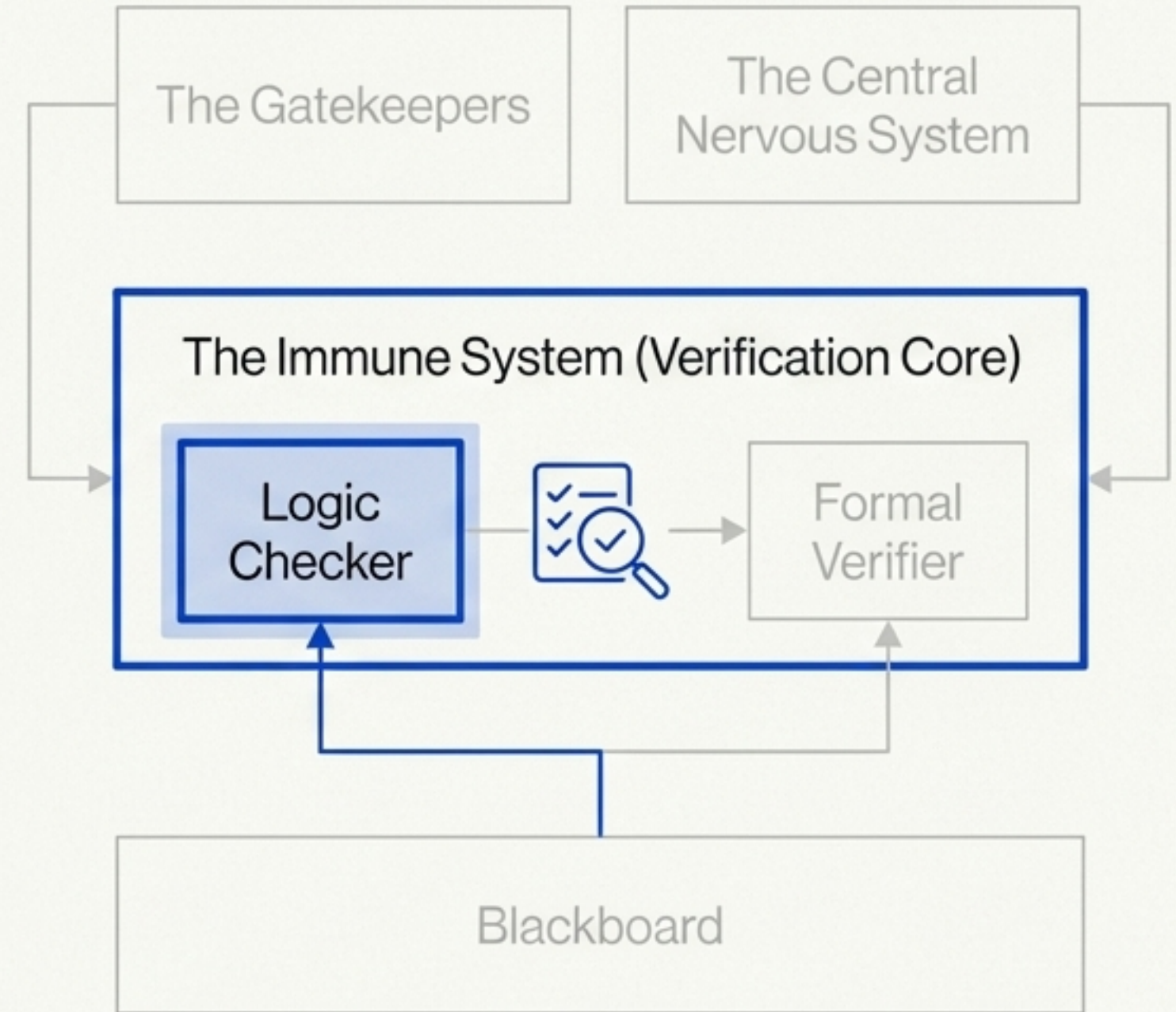
Step 4: The Immune System Intercepts the Solution

Objective

Implement the **Ax-Prover** pattern immediately. No result from any solver is allowed to leave the system until this team stamps it “Approved.” This is the primary defense against hallucination.

First Line of Defense: The Logic Checker

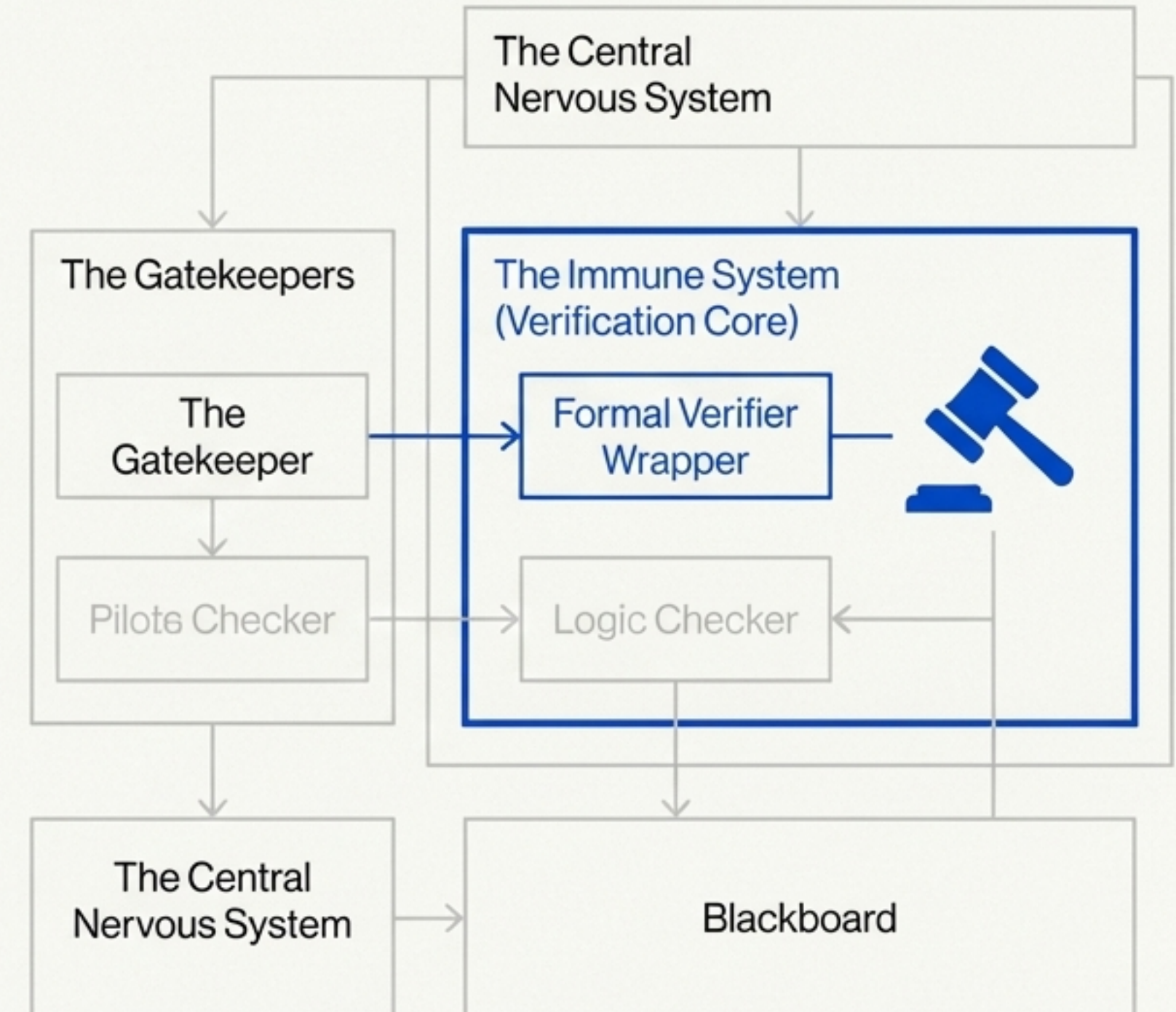
- A rule-based agent that inspects the logical flow of the solution.
- It checks for “illegal moves” and addresses the “Assumption Gap.”
- **Examples:**
 - Dividing by a variable without confirming it is non-zero.
 - Applying a real-domain logarithm to a negative number.



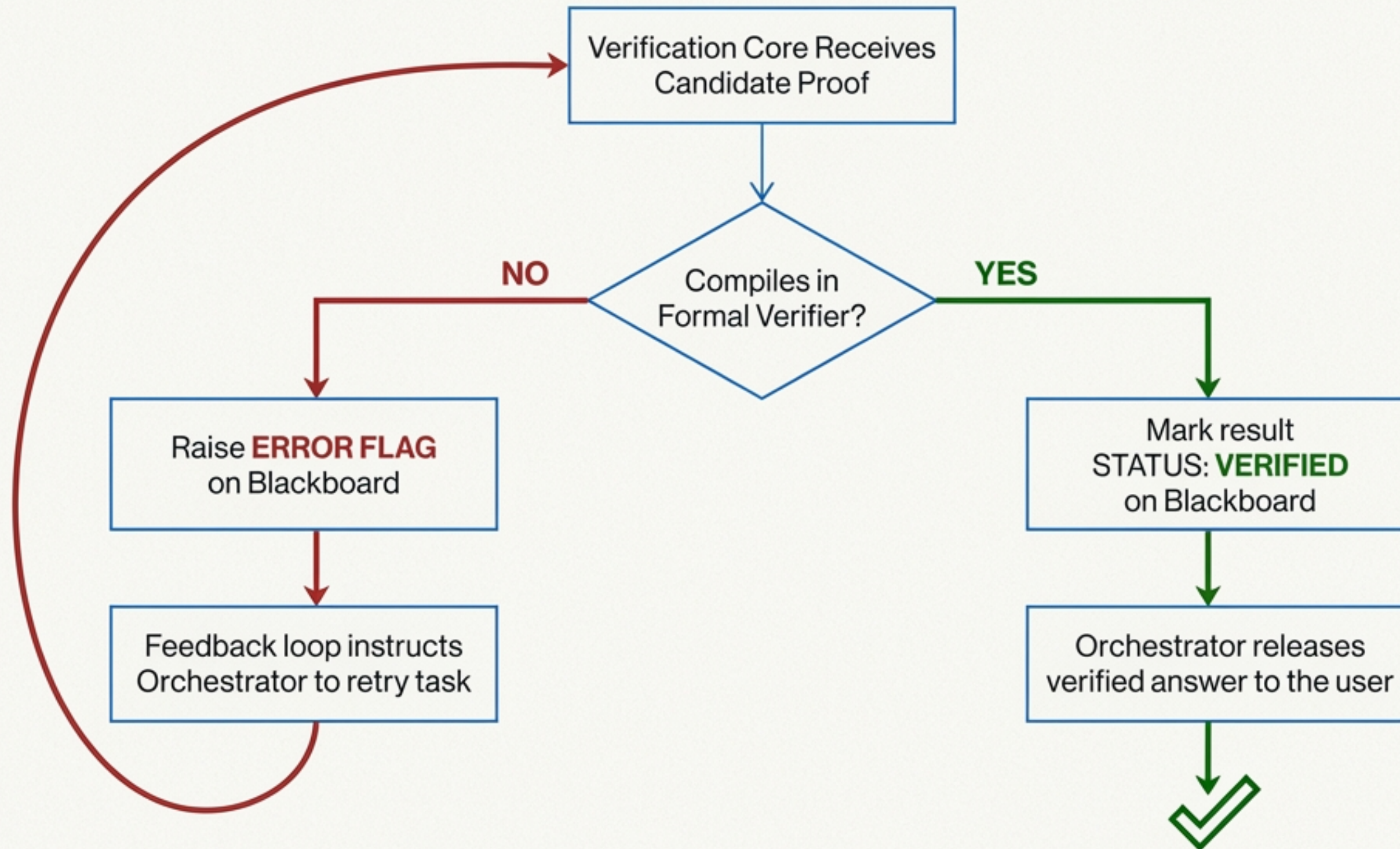
The Formal Verifier Provides the Ultimate Guarantee

Second Line of Defense: The Formal Verifier Wrapper

- An agent that wraps a formal theorem prover (e.g., **Lean 4**, **Coq**, or **Isabelle**).
- **Process:** It treats the Pilot Solver's output as a “candidate proof” and attempts to compile it within the formal system.
- This is the final, absolute check against hallucination. If the solution can be formally proven, it is correct.

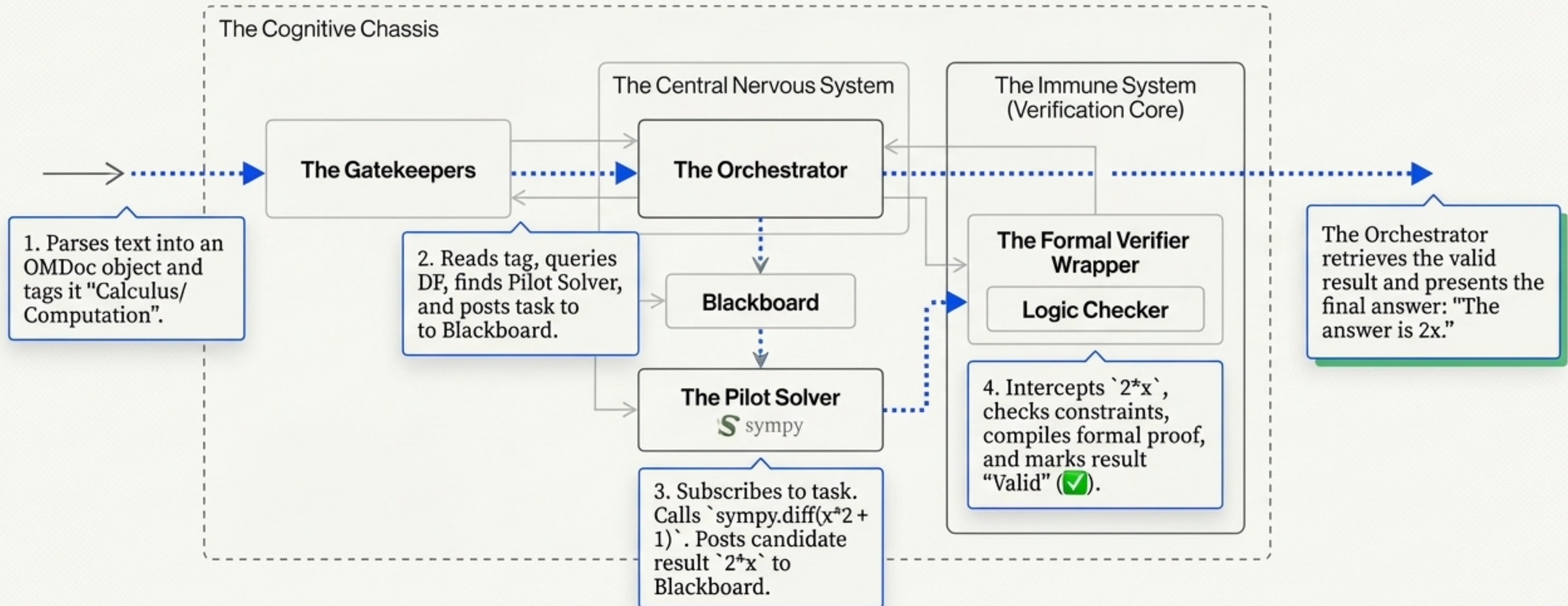


The Neural-Symbolic Loop: Success or Retry



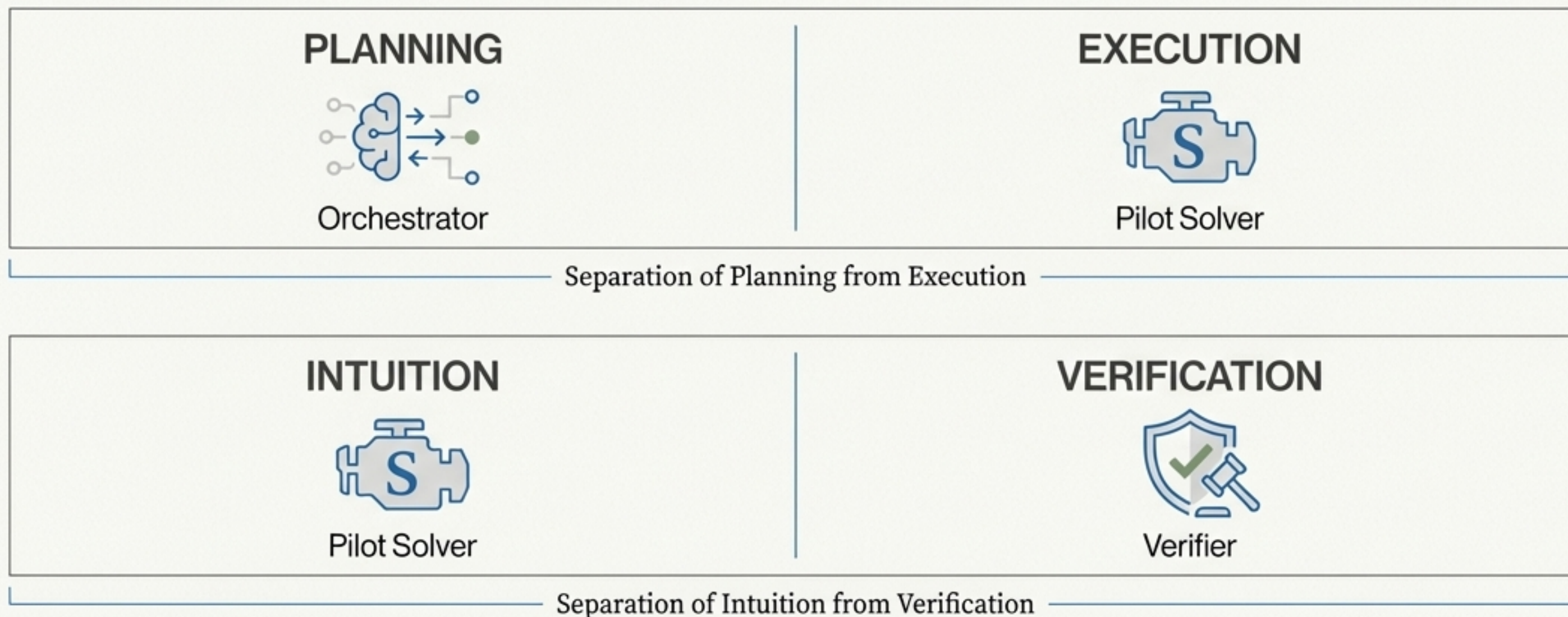
Step 5: The 'First Breath' of the Cognitive Chassis

The Journey of "Calculate the derivative of $x^2 + 1$ "



The Phase 1 Deliverable: Architecturally Complete, Mathematically Limited

What Has Been Built: A system that is now **architecturally complete**. The foundational reasoning loop is functional and verified.



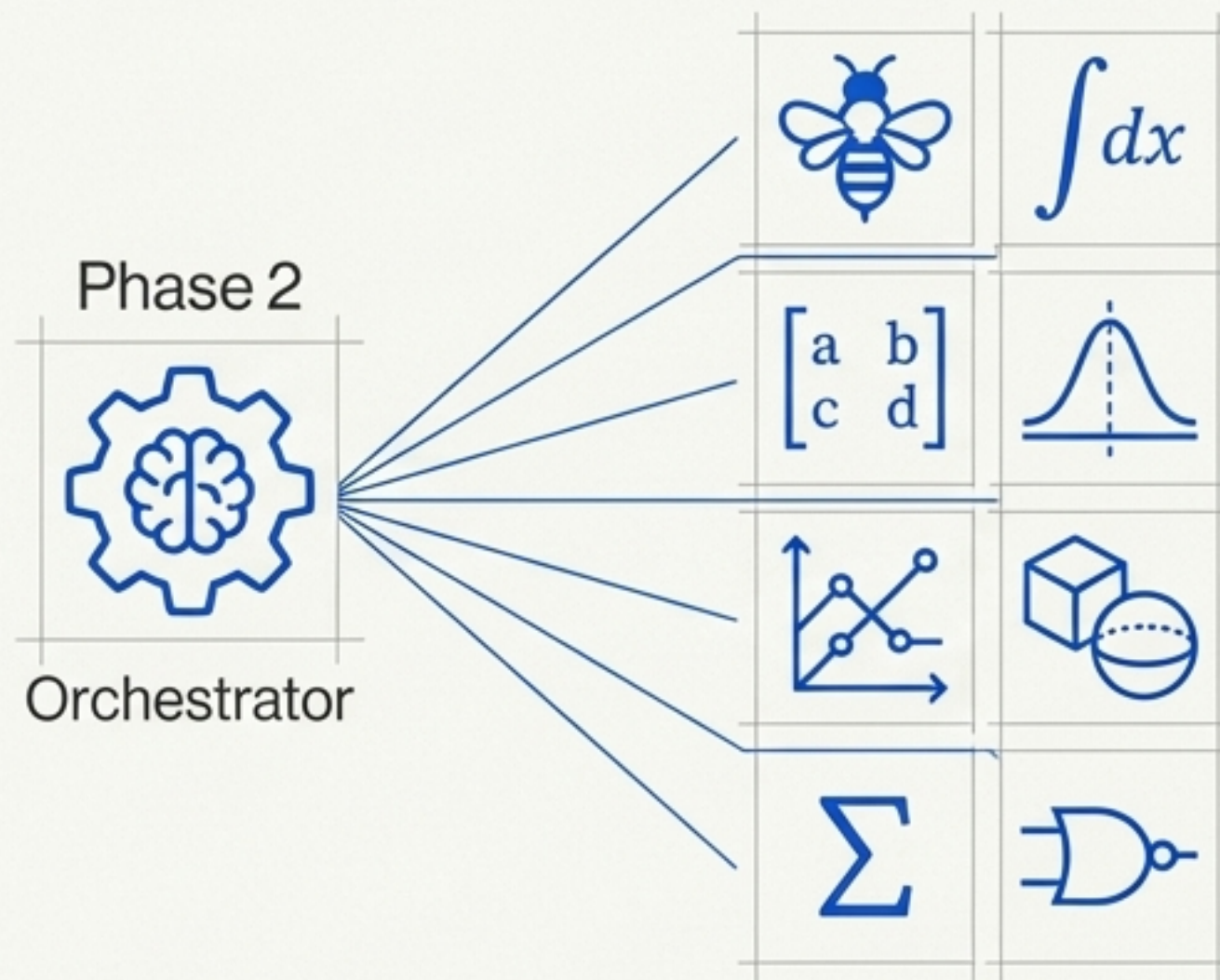
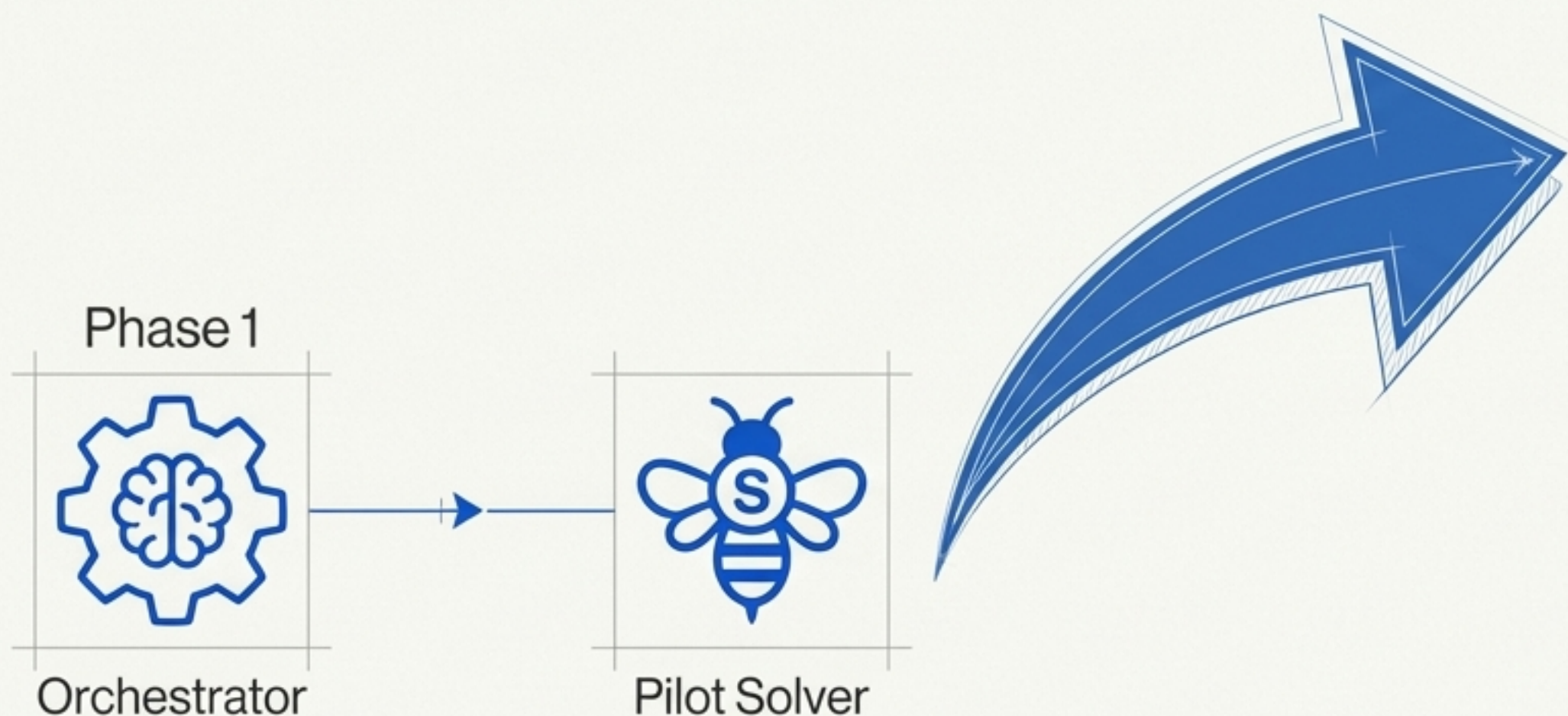
The Deliberate Limitation

The system is currently 'mathematically limited' because its capability is bound to the SymPy library. This was by design, to perfect the chassis before adding more engines.

From a Single Engine to a Specialized Workforce

The Foundation is Set

The Cognitive Chassis is not just a demo; it's a scalable foundation. The Orchestrator's dynamic routing and the strict verification protocols are designed for expansion.



The Road to Phase 2

The next step is to replace the single 'Pilot Solver' with a **massive, specialized workforce** of Tier 2 agents.

- With the chassis built and the verification loop in place, the system is now ready to safely and reliably scale its mathematical capabilities.



Q&A

Thank You

